Software Engineering Internship Assignment Report

Project Title: BookVault - Full-Stack Library Management System

Name: Yasiru De Silva

GitHub Repository: https://github.com/Yasiru21/BookVault

Technology Stack: C# .NET 8 Web API | React 18 + TypeScript | SQLite + Entity Framework Core

Table of Contents

1. Introduction

This report documents the complete development process, technical decisions, and implementation details of BookVault, a full-stack Library Management System built as part of the software engineering internship assignment. The system allows users to browse, search, and manage a catalog of books through a modern web interface backed by a RESTful API.

The primary requirements were to implement full CRUD (Create, Read, Update, Delete) operations on both the backend and frontend, integrate a SQLite database using Entity Framework Core, and deliver a functional, well-documented application. Beyond these core requirements, the project was extended with JWT-based user authentication, a user profile system, real-time search, and filtering, and a professionally styled, fully responsive user interface.

The development followed an iterative approach. The backend API was built first to establish a stable data layer. Once the API endpoints were verified through Swagger, the frontend was developed layer by layer, starting with data fetching and progressing through routing, forms, and finally authentication.

2. Technology Stack and Justification

2.1 Backend:

- **C# with .NET 8 Web API** - The framework required by the assignment. .NET 8 is the current long-term support release and offers excellent performance for RESTful services.
- **Entity Framework Core 9** - Used as the Object-Relational Mapper (ORM) to interact with the database. EF Core allows the database schema to be defined directly in C# code as model classes, eliminating the need to write raw SQL for standard operations.
- **SQLite** - A lightweight, file-based database engine. Chosen because it requires no separate server installation, making the application fully portable. Any developer who clones the repository can run the application without setting up a database server.
- **BCrypt.Net** - Used to securely hash user passwords. BCrypt is a password-hashing function designed to be computationally expensive and resistant to brute-force attacks.
- **System.IdentityModel.Tokens.Jwt** - The standard .NET library for generating and validating JSON Web Tokens.
- **Swashbuckle (Swagger)** - Auto-generates interactive API documentation from the code, accessible in the browser at /swagger.

2.2 Frontend:

- **React 18 with TypeScript** - A modern component-based UI library paired with static type checking.
- **Vite** - A fast build tool and development server for React applications.
- **React Router v6** - Handles client-side navigation and protected routes.

- **Axios** - HTTP client for making API requests, configured with request and response interceptors.
- **react-hook-form + Zod** - Used together for form state management and schema-based input validation.
- **CSS Modules** - Standard CSS files scoped to individual components, avoiding global style conflicts.
- **Lucide React** - A consistent, clean icon library used throughout the interface.

3. System Architecture

The application follows a decoupled client-server architecture. The backend is a purely stateless REST API, and the frontend is a Single Page Application (SPA) that communicates with it over HTTP.

3.1 Backend Layered Architecture

The backend is organized into four distinct layers, strictly following the Separation of Concerns principle.

Controllers Layer - The BooksController and AuthController are responsible only for receiving HTTP requests, passing data to the service layer, and formatting the HTTP response. They contain no business logic. For example, the BooksController.CreateBook method simply passes the validated DTO to _bookService.CreateBookAsync() and returns a 201 created response with the result.

Service Layer (Business Logic) - BookService and AuthService contain all the business logic. They are defined behind interfaces (IBookService, IAuthService), which is a direct application of the Dependency Inversion Principle. This design means the controllers depend on the abstraction (the interface) rather than the concrete implementation. This makes the system significantly easier to maintain and evaluate, because the concrete class can be swapped out or mocked without changing any controller code.

Data Access Layer - LibraryDbContext extends EF Core's DbContext and serves as the gateway to the SQLite database. All database queries are performed here using EF Core's LINQ-based query API.

DTO Layer (Data Transfer Objects) - A deliberate decision was made to use separate DTO classes instead of passing raw database entities between layers. There are three types of DTOs used:

- BookResponseDto - controls what is returned to the client. The UpdatedAt and CreatedAt timestamps are included, but internal fields are not exposed.
- CreateBookDto - defines the exact payload accepted for creating a book, with validation annotations.
- UpdateBookDto - defines the exact payload accepted for updates.

This pattern prevents "over-posting" attacks, where a malicious user might submit additional fields in a request to modify data, they should not have access to. Since the API only reads the fields defined in the DTO, extra submitted fields are simply ignored.

3.2 Frontend Architecture

The frontend is organized into five main directories, each with a single clear responsibility:

- **components/** - Small, reusable pieces of UI (Navbar, BookCard, ConfirmModal, BookForm, Footer).
- **pages/** - Full-page views assembled from components (BookListPage, BookDetailPage, AddBookPage, EditBookPage, LoginPage, RegisterPage, ProfilePage, ContactPage, FeaturesPage, AboutPage).
- **context/** - Global application state managed using React's Context API.
- **services/** - All Axios HTTP calls are abstracted here, completely separated from the UI logic.
- **types/** - TypeScript interface definitions that function as the contract between the frontend and backend.

4. Database Design

The SQLite database contains two tables managed by EF Core.

4.1 Book Entity

The Book model contains the following fields:

- Id (int) - Auto-incremented primary key managed by EF Core.
- Title (string, required, max 200 chars)
- Author (string, required, max 150 chars)
- Description (string, required, max 2000 chars)
- ISBN (string, optional, max 20 chars, unique when provided)
- Genre (string, optional, max 100 chars)
- PublishedYear (int, optional)
- CreatedAt (DateTime, UTC) - Records when the entry was added.
- UpdatedAt (DateTime, UTC) - Refreshed on every save operation.

Database-level indexes are defined using EF Core's Fluent API in OnModelCreating. Indexes are created on Title and Author to speed up search queries, and a partial unique index is placed on ISBN (the "partial" aspect means the uniqueness constraint only applies to non-null values, allowing multiple books to have no ISBN).

4.2 User Entity

The User model contains:

- Id (int) - Auto-incremented primary key.
- Username (string, required, max 50 chars, unique at DB level)
- Email (string, required, max 200 chars, unique at DB level)
- PasswordHash (string, required) - The BCrypt hash of the user's password. The raw password is never stored anywhere.
- Role (string, default "User")
- CreatedAt (DateTime, UTC)

4.3 Database Initialization and Seeding

The application uses dbContext.Database.EnsureCreated() on startup (in Program.cs) to automatically create the database schema from the EF Core model if it does not already exist. This means the SQLite file (library.db) does not need to be included in the repository. When a new developer clones the project and runs the backend for the first time, the database file is created automatically.

Seed data is defined using EF Core's HasData() method inside OnModelCreating. This populates the database with 16 sample books across 10 genres (Technology, Fantasy, Sci-Fi, Romance, Fiction, Biography, Self-Help, Mystery, History, Psychology) on the very first run.

5. Backend Implementation - CRUD Operations in Detail

All five CRUD operations are implemented as asynchronous methods throughout the entire stack to avoid blocking thread-pool threads, which is critical for scalability in a web application.

5.1 Read - GET /api/books

The GetAllBooksAsync method in BookService builds a LINQ query using EF Core's deferred execution model. The query is constructed step by step - first applying the search filter (case-insensitive substring match on Title or Author), then the genre filter - but no database call is made until the query is finalized. The total count is fetched in one database round-trip, and then .Skip((page-1) * pageSize).Take(pageSize) is applied for server-side pagination. The AsNoTracking() extension is used on read-only queries, which improves performance by telling EF Core not to track the returned entities for change detection.

5.2 Read - GET /api/books/{id}

Uses FindAsync(id), which directly uses the primary key index for the most efficient single-record lookup. Returns null if not found, which the controller converts to a 404 Not Found response.

5.3 Create - POST /api/books

This endpoint is decorated with [Authorize], meaning a valid JWT must be provided. The CreateBookDto is passed in from the request body. The [ApiController] attribute on the

controller automatically validates the DTO's DataAnnotations and returns a 400 Bad Request if validation fails, before the action method even executes. On success, the controller returns a 201 Created response with the Location header pointing to the new resource's URL (/api/books/{newId}).

5.4 Update - PUT /api/books/{id}

It also requires authorization. The service fetches the existing entity using FindAsync, applies the new field values from the DTO, updates the UpdatedAt timestamp, and calls SaveChangesAsync(). EF Core's change-tracking mechanism detects which properties were modified and generates an efficient SQL UPDATE statement. Returns 404 if the book does not exist.

5.5 Delete - DELETE /api/books/{id}

Requires authorization. Fetches the entity, calls _context.Books.Remove(book), and persists with SaveChangesAsync(). Returns 204 No Content on success (the standard HTTP response for a successful delete with no response body) or 404 if not found.

5.6 Genres - GET /api/books/genres

A dedicated endpoint that queries the database for a distinct, alphabetically sorted list of all genres currently in use. This is consumed by the frontend to dynamically populate the genre filter pill buttons.

6. Authentication System

6.1 Registration

When a user submits the registration form, the AuthController receives a RegisterDto containing the username, email, and password.
The AuthService.RegisterAsync method first checks the database for any existing user with the same email or username. If a duplicate is found, it returns null, which the controller converts to a 409 Conflict HTTP response.

If the user is new, BCrypt hashes the password with a work factor of 12 before creating the User entity. The work factor controls how computationally expensive the hash operation is. A factor of 12 means the hash takes approximately 250-500ms to compute, which makes brute-force attacks impractical. The raw password is discarded immediately and never stored anywhere.

6.2 Login

The LoginAsync method looks up the user by email and then calls BCrypt.Verify() to compare the submitted password against the stored hash. If verification passes, a JWT is generated.

6.3 JWT Token Generation

The JWT is generated in GenerateAuthResponse. The token payload (claims) includes the user's ID, email, username, and role. The token is set to expire after 24 hours. The signed token string is returned to the client alongside the user's display information.

6.4 Token Validation and Protected Routes

In Program.cs, the JWT authentication middleware is configured with parameters that validate the issuer, audience, token lifetime, and signing key on every incoming request that carries a Bearer token. Any endpoint decorated with [Authorize] will automatically reject requests without a valid token with a 401 Unauthorized response.

7. Frontend Implementation

7.1 Global Authentication State

AuthContext.tsx implements the React Context API to provide authentication state globally. The AuthProvider wraps the entire application, and any component can access the current user via the useAuth() hook. On login, the JWT and user profile object are stored in localStorage so the session persists across browser refreshes. On logout, both items are cleared. The context exposes isAuthenticated as a derived boolean, simplifying conditional rendering throughout the application.

7.2 Axios Configuration and Interceptors

The central api.ts file creates a configured Axios instance with the base URL set to /api. Two interceptors are registered:

- **Request Interceptor** - Before every outgoing HTTP request is sent, this interceptor reads the JWT from localStorage and, if found, appends it to the Authorization header as Bearer <token>. This means every component that calls a service function automatically sends the authentication token without any extra code.

- **Response Interceptor** - If any response comes back with a 401 Unauthorized status code, this interceptor automatically clears the stale token from localStorage and redirects the user to the login page. This manages the edge case where a token has expired server-side while the user was already in an active session.

7.3 Typed Service Layer

bookService.ts and authService.ts contain strongly typed wrapper functions around the Axios instance. Each function returns a typed Promise (e.g., Promise<PagedResult<Book>>, Promise<Book>). This means that when a component calls getBooks(), TypeScript guarantees the shape of the returned data. Any mismatch between the frontend types and the backend's actual JSON response is caught during development.

7.4 Form Validation

All forms use react-hook-form for state management, integrated with Zod for schema-based validation. A Zod schema is defined for each form (e.g., bookSchema, loginSchema) that specifies exact type rules, minimum/maximum lengths, and required fields. react-hook-form connects this schema to the form inputs and manages error state. Validation messages are shown inline next to each field in real-time as the user types, without any network requests.

7.5 Protected Routes

A ProtectedRoute component wraps routes that require authentication (Add Book, Edit Book). When an unauthenticated user attempts to navigate to /books/new or /books/:id/edit, the component checks isAuthenticated from the auth context and redirects them to /auth/login. For editing and deleting actions on book cards, the same check is applied via an onClick handler - if the user is not authenticated, clicking the button redirects to login rather than hiding the button altogether.

8. Additional Features Implemented

- **Real-time Search and Pagination -** The book catalog supports a debounced search input that filters by title or author. A debounce delay of 400ms is applied so that the API is not called on every single keystroke. Genre filter pills are dynamically populated from the GET /api/books/genres endpoint. Server-side pagination returns 9 books per page.
- **Book Detail View -** Clicking a book card navigates to a dedicated detail page that displays all book metadata including a reading time estimate (calculated from description length) and the book's age in years.
- **User Profile Page -** A protected profile page accessible from the navbar displays the logged-in user's username, email, and account creation date.
- **Features, About, and Contact Pages -** Informational pages were created to provide a complete, polished application experience. The Contact page includes all personal social links and an FAQ accordion section.
- **Dark/Light Mode Toggle -** A theme toggle in the navbar switches between a dark glassmorphism theme and a light blue theme. The selected theme persisted in localStorage.

9. Error Handling and Validation Strategy

Validation operates at two independent layers, providing redundancy.

- **Backend validation** - Managed by .NET's DataAnnotations system. Fields on CreateBookDto and UpdateBookDto are annotated with [Required], [MaxLength], [Range], and [EmailAddress]. The [ApiController] attribute automatically enforces these rules and returns a structured 400 Bad Request response with field-level error

messages if any constraint is violated. This prevents invalid data from ever reaching the service or database layer.

- **Frontend validation** - Managed by Zod schemas. The frontend validation runs entirely on the client side and provides immediate feedback without waiting for a round-trip to the server. However, it is important to note that frontend validation is a usability feature, not a security measure. The backend validation is the definitive line of defense.

- **Global error handling -** API errors beyond validation (such as network failures or 500 Internal Server Error) are caught in try-catch blocks within page components and displayed to the user as toast notifications. This prevents the application from silently failing or showing raw error objects.

10. Code Documentation

A consistent documentation standard was maintained throughout the codebase.

On the backend, all public classes, methods, and key properties are documented using C# XML documentation comments (/// <summary>). This serves two purposes: it acts as documentation for developers reading the code, and it is automatically picked up by Swagger to generate human-readable API descriptions for each endpoint.

On the frontend, JSDoc-style block comments (/** */) are used for service functions and context providers. Inline comments are used to explain non-obvious logic, such as the purpose of the Axios interceptors, the debounce mechanism, and the CSS custom property cascade. The types/index.ts file is particularly well-commented since it defines the core data contracts used across the entire application.

11. Development Challenges and Solutions

Challenge 1: Managing Stale UI State After Mutations

After deleting a book, the UI needed to update instantly without a full page reload. React's state model means the list of books displayed on the screen is derived from a state variable. When a deletion succeeds, the local state array is filtered to remove the deleted book's ID: setBooks(prev => prev.filter(b => b.id !== id)). This optimistic UI update provides an instant response, even before any refetch, making the application feel fast and responsive.

Challenge 2: JWT Session Expiry

A JWT token has a defined expiry time (24 hours in this system). If a user's token expires while they are in an active browser session, their next API request will return a 401 Unauthorized error. Without handling this, the user would see a confusing error message. The global Axios response interceptor solves this elegantly: it listens to any 401 responses across all API calls, automatically clears the expired token from localStorage, and redirects the user to the login page with no additional code required in any individual component.

Challenge 3: Coordinating the Full-Stack Development Environment
Running a .NET API and a Vite dev server simultaneously require two separate terminal processes. Carefully documenting the startup order in the README (backend first, then frontend) was important, because the frontend's API calls will fail if the backend is not running. This was documented clearly with exact commands for both Windows and cross-platform environments.

12. Key Insights Gained

The most significant technical insight from this project was understanding the value of layered architecture in practice, not just in theory. When the decision was made later in development to add JWT authentication, the existing BooksController required almost no changes because all business logic was isolated in the service layer. Adding the [Authorize] attribute to three action methods was all that was needed on the controller side. The authentication service was entirely independent, added without touching any existing code.

The use of TypeScript interfaces as a shared contract between frontend and backend also proved extremely valuable. Whenever the backend DTO changed (such as adding a new field to BookResponseDto), the TypeScript compiler immediately flagged every frontend location that needed to be updated, rather than allowing those mismatches to become silent runtime bugs.

Finally, working through the full development cycle of a real application - from database schema design to pixel-level UI styling - reinforced that software engineering is a deeply interconnected discipline. Decisions made in the database schema (like adding indexes) have direct and measurable impacts on API response times, which in turn affect the frontend user experience.

13. Running the Application

Prerequisites

- .NET 8 SDK
- Node.js 18+

Steps

bash
Step 1: Clone the repository
git clone https://github.com/Yasiru21/BookVault.git
cd BookVault

```
# Step 2: Start the backend (in Terminal 1)
cd backend/LibraryAPI
dotnet restore
dotnet run
# API runs at http://localhost:5000
# Swagger UI at http://localhost:5000/swagger

# Step 3: Start the frontend (in Terminal 2)
cd frontend
npm install
npm run dev
# App runs at http://localhost:5173
```

The SQLite database is created automatically on the first backend run. No database setup is required.

The End